Cahier des charges — Projet S4 2025–2026

Introduction

La comparaison et la détection de similarité entre fichiers constituent un enjeu majeur dans de nombreux domaines de l'informatique, tels que la détection de plagiat, l'identification de doublons, l'analyse de versions de documents, ou encore l'optimisation du stockage de données.

Ce projet vise à concevoir un **moteur logiciel modulable** capable d'évaluer de manière fiable le degré de similarité entre fichiers texte, code source et images, tout en garantissant de bonnes performances et une utilisation maîtrisée des ressources.

État de l'art condensé

1. Comparaison naïve caractère par caractère

Principe : comparer directement les fichiers caractère par caractère.

Capacités :

- Implémentation simple
- Résultats exacts pour des fichiers strictement identiques ou très proches

Limites :

- Très sensible aux modifications mineures (espaces, retours à la ligne)
- Peu pertinente pour mesurer une similarité globale
- Inefficace pour des fichiers de grande taille

---

2. Distances d'édition (Levenshtein, Damerau-Levenshtein)

Principe : mesurer le nombre minimal d'opérations nécessaires pour transformer un fichier en un autre.

Capacités :

- Mesure précise des différences
- Bien adaptée aux fichiers courts
- Fournit une valeur quantitative exploitable

Limites :

- Complexité algorithmique élevée (souvent $O(n^2)$)
- Mauvaise scalabilité pour de grands fichiers

- Consommation mémoire importante

---

3. Approches basées sur les k-grams (shingles)

Principe : découper les fichiers en sous-chaînes de longueur fixe (k-grams) et comparer les ensembles via la similarité de Jaccard.

Capacités :

- Robuste aux modifications locales
- Bon compromis précision / performance
- Largement utilisée dans la détection de plagiat

Limites :

- Choix du paramètre k critique
- Sensible au bruit si mal paramétrée
- Peut générer de grands ensembles de données

---

4. Techniques de hachage et d'empreintes (MinHash)

Principe : représenter un fichier par une signature compacte via des fonctions de hachage.

Capacités :

- Comparaison rapide
- Faible consommation mémoire
- Adaptée aux grands volumes de données

Limites :

- Approche probabiliste (approximation)
- Risque de faux positifs / faux négatifs
- Moins précise que les méthodes exactes

---

5. Analyse spécifique du code source (tokenisation)

Principe : analyser les fichiers au niveau lexical (tokens) plutôt que caractère par caractère.

Capacités :

- Insensible au formatage
- Détection de similarité logique
- Adaptée aux fichiers de code source

Limites :

- Implémentation plus complexe
- Dépendante du langage analysé
- Hors périmètre strict du texte brut

---

6. Comparaison d'images

Principe : représenter les images sous forme de matrices de couleurs ou d'histogrammes pour évaluer leur similarité.

Algorithmes possibles :

- Pixel à pixel pour des images très proches
- Histogramme de couleurs (RGB ou HSV) avec distance Euclidienne ou Chi-squared
- Descripteurs avancés (SIFT, ORB) pour tolérer rotation ou redimension

Capacités :

- Comparaison efficace d'images de tailles et formats différents
- Robuste aux petits changements ou compressions
- Possibilité d'utiliser pHash pour accélérer les comparaisons

Limites :

- Pixel à pixel sensible aux transformations
- Algorithmes avancés complexes à implémenter
- Comparaison approximative pour certaines méthodes de hachage

---

Technologie retenue et justification

Approche modulaire et hybride :

- **Texte et code :** k-grams combinés à des techniques de hachage, mesure de similarité Jaccard et empreintes MinHash

- **Images :** histogramme de couleurs et pHash pour comparaison rapide, pixel à pixel ou descripteurs avancés en option

Justification :

- Bon équilibre précision / performance
- Scalabilité adaptée aux fichiers de tailles et types variés
- Méthodes reconnues et éprouvées
- Compatible avec une architecture modulaire et les contraintes du langage Rust
- Maîtrise de la complexité algorithmique et de la consommation des ressources

---

Répartition des charges entre les membres

Membre	Rôle principal	Tâches principales	Pourcentage estimé
Ayoub El Asri	Algorithmes de similarité	- Implémentation pour texte/code/images (k-grams, Jaccard, MinHash, histogramme/pHash) - Analyse des performances et précision - Comparaison des approches - Tests unitaires - Contribution à l'état de l'art et justification technique	40 %
Lucas Lefranc	Architecture logicielle et intégration	- Conception architecture modulaire du moteur - Gestion entrées/sorties fichiers - Optimisation performance/mémoire - Interface moteur ↔ site - Tests globaux et intégration continue - Déploiement Linux	35 %
Maywenn Koussawo	Interface Web	- Conception et développement site web - Gestion upload et affichage des résultats - Interaction moteur logiciel	25 %

Membre	Rôle principal	Tâches principales	Pourcentage estimé
		- Tests fonctionnels - Documentation utilisateur	

Note : les pourcentages reflètent l'effort relatif estimé sur l'ensemble du projet.

Planning détaillé de réalisation

Période	Objectifs principaux	Tâches / Livrables	Responsable(s)
Semaines 1–4	Analyse et conception	- Étude complète état de l'art - Conception architecture initiale - Découpage modules algorithmes et site web - Choix des technologies	Tous
Semaines 5–8	Prototype initial	- Implémentation k-grams + Jaccard pour texte/code - Début de la structure du moteur Rust - Création site web minimal - Tests unitaires initiaux	Ayoub / Lucas / Maywenn
Semaine 9	**Soutenance intermédiaire**	- Présentation du prototype - Démo site web avec upload et résultats simplifiés - Explication des choix algorithmiques et architecture	Tous
Semaines 10–12	Développement avancé	- Ajout MinHash pour texte/code - Intégration modules images (histogramme/pHash) - Optimisation performances	Ayoub / Lucas
Semaines 13–16	Intégration complète	- Finalisation moteur texte/code/images - Intégration site web ↔ moteur - Tests globaux et validation	Tous

Période	Objectifs principaux	Tâches / Livrables	Responsable(s)
Semaines 17–18	Préparation soutenance finale	- Documentation complète (utilisateur/technique) - Benchmarks et comparaisons algorithmes - Déploiement Linux et packaging - Préparation démo finale	Tous
Semaine 19	**Soutenance finale**	- Démonstration complète du logiciel - Analyse critique des résultats	Tous

---

Conclusion

Le projet a pour objectif de développer un moteur logiciel capable de détecter la similarité entre fichiers texte, code source et images, en combinant **performance, précision et modularité**.

Chaque type de fichier dispose de son propre module, permettant de sélectionner dynamiquement l'algorithme le plus adapté selon le type et la taille des fichiers, assurant ainsi **scalabilité et maintenabilité optimales**.

La répartition claire des rôles — Ayoub El Asri pour les algorithmes, Lucas Lefranc pour l'architecture et l'intégration, et Maywenn Koussawo pour le site web — garantit un **suivi efficace et équilibré** du projet.

Le choix des technologies retenues, notamment **Rust pour le moteur**, **k-grams/Jaccard/MinHash pour texte/code** et **histogramme/pHash pour images**, offre un **équilibre entre robustesse, performance et simplicité de mise en œuvre**, répondant aux objectifs pédagogiques et techniques.

Ce cahier des charges constitue donc une **référence pour toutes les phases de conception, prototypage et finalisation** du logiciel.